THE AI TRADING PLAYBOOK

PRO EDITION

Hybrid AI Infrastructure, Autonomous Trading Systems,

and Deployment Frameworks

A Complete Technical Guide for Building
Institutional-Grade AI Trading Infrastructure

2026 Edition

TABLE OF CONTENTS

Chapter 1: Introduction
- What You Will Build
- Who This Playbook Is For
- Prerequisites

Chapter 2: Hybrid AI Trading Architecture
- The Case for Hybrid Infrastructure
- Local Systems (Latency-Sensitive Tasks)
- Cloud AI Models (Reasoning Tasks)
- Architecture Diagram
- Communication Patterns

Chapter 3: Windows + WSL Trading Infrastructure
- 3.1 Recommended Hardware Specifications
- 3.2 Installing WSL2
- 3.3 NVIDIA GPU Setup for WSL2
- 3.4 Python Environment Setup
- 3.5 Installing Ollama for Local LLM Inference
- 3.6 Node.js and TypeScript Setup
- 3.7 WSL2 Networking Configuration
- 3.8 SSH Access Between Windows and WSL2

Chapter 4: Mac AI Trading Infrastructure
- 4.1 Why Apple Silicon for AI Trading
- 4.2 macOS Environment Setup
- 4.3 Python Environment for Mac
- 4.4 Ollama on macOS
- 4.5 Tailscale for Secure Multi-Node Communication
- 4.6 Mac as a Monitoring Dashboard

Chapter 5: Strategy Engine
- 5.1 Strategy Types
  - Mean Reversion
  - Momentum Breakouts
  - Funding Rate Arbitrage
  - Options Income
- 5.2 Multi-Strategy Signal Aggregation

Chapter 6: AI Decision Layer
- 6.1 Signal Evaluation Pipeline
- 6.2 Cloud Fallback for Complex Analysis
- 6.3 Tiered Model Routing

Chapter 7: RAG Market Memory
- 7.1 Setting Up ChromaDB for Trade Memory

7.2 Integrating Memory with AI Decisions
Chapter 8: Hyperliquid Trading Bot Architecture
8.1 Bot Architecture
8.2 Hyperliquid API Integration
8.3 Scalper Configuration
Chapter 9: Market Maker Strategy
9.1 Core Market Making Logic
Chapter 10: Volatility Regime Detection
10.1 Regime Classification
10.2 Strategy Selection by Regime
Chapter 11: Portfolio Risk Engine
11.1 Risk Parameters
11.2 Kill Switch Implementation
Chapter 12: Monitoring and Observability
12.1 What to Monitor
12.2 WhatsApp Notification Integration
12.3 Remote Command System
Chapter 13: Scaling AI Trading Systems
13.1 Scaling Strategies
13.2 Multi-Agent Architecture with pm2
Chapter 14: Conclusion
Key Takeaways
Next Steps

Chapter 1: Introduction

Artificial intelligence is rapidly transforming financial markets. Retail traders now have access to tools that were once limited to hedge funds, including machine learning models, GPU compute infrastructure, and automated trading systems.

This playbook provides a complete technical blueprint for building a hybrid AI trading stack that combines local inference systems, cloud reasoning models, algorithmic trading engines, and automated risk controls. The goal is to design a robust trading platform capable of continuously monitoring markets and executing strategies with minimal human intervention.

What You Will Build

The system described in this guide integrates several critical components into a unified platform:

- Market data ingestion pipelines for real-time and historical data across multiple venues
- Quantitative strategy engines implementing mean reversion, momentum, funding rate, and options strategies
- LLM-based reasoning layers using both local (Ollama) and cloud (Claude, GPT) models
- Retrieval-Augmented Generation (RAG) market memory for historical pattern recall
- Automated execution engines for Hyperliquid perps, Polymarket, and MES futures options
- Portfolio-level risk management with drawdown limits, exposure caps, and kill switches
- Monitoring and reliability infrastructure with WhatsApp/Telegram alerting

Together these components form a complete AI trading platform that can evolve as markets change.

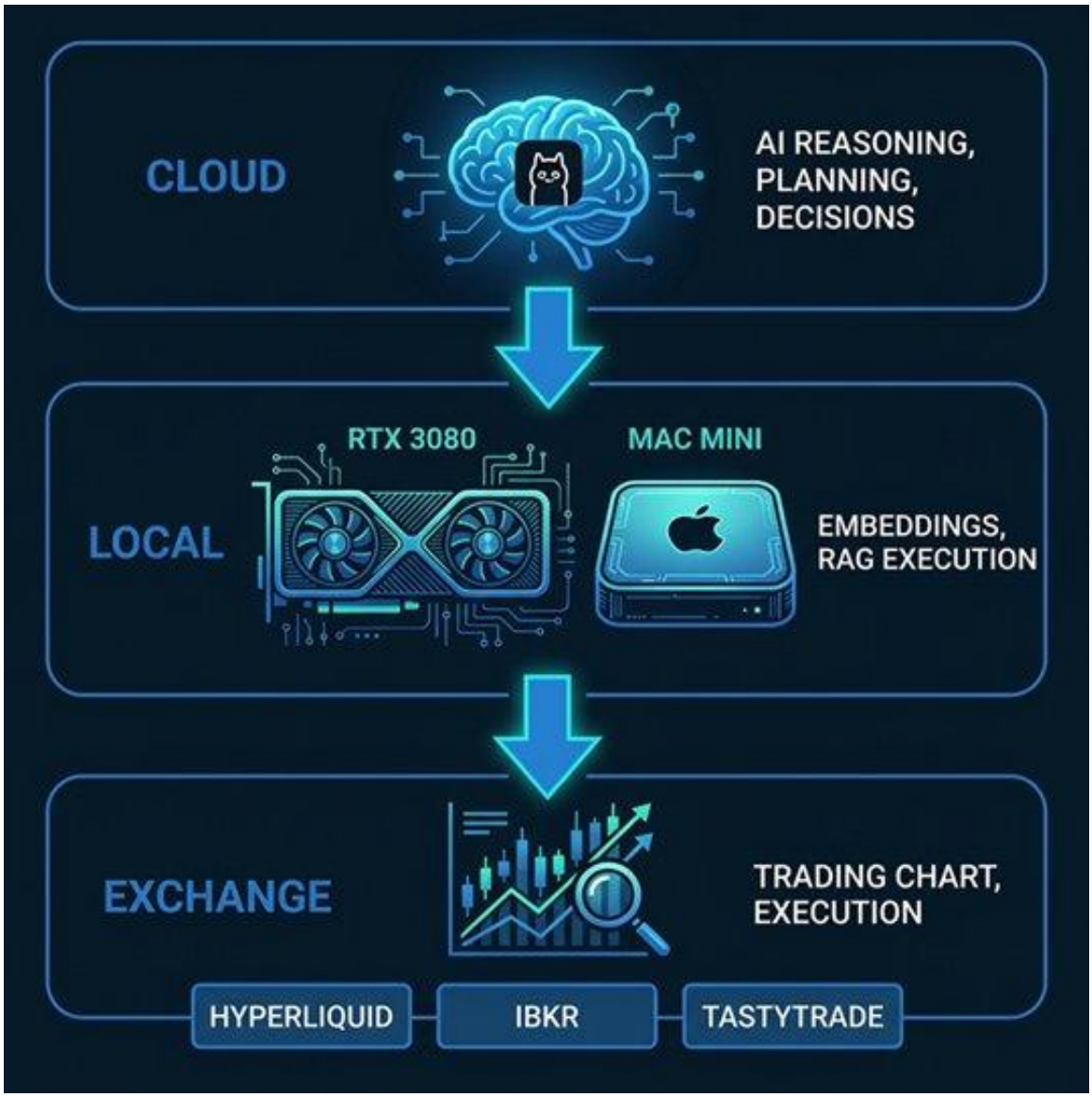
Who This Playbook Is For

This guide is designed for technically inclined traders who are comfortable with Python, JavaScript/TypeScript, and command-line tools. You should have a basic understanding of financial markets and derivatives. Prior experience with machine learning is helpful but not required — we will build everything from the ground up.

Prerequisites

Skill Area	Minimum Level
Python	Intermediate — comfortable with async, classes, pip/venv
JavaScript/TypeScript	Intermediate — Node.js, npm, async/await patterns
Command Line	Comfortable with bash, SSH, package managers
Trading Knowledge	Understanding of order types, leverage, risk management basics
Git	Basic version control for managing your codebase

Chapter 2: Hybrid AI Trading Architecture



A hybrid architecture divides responsibilities between local compute and cloud AI models. This separation is not merely a design preference — it is a practical necessity driven by latency requirements, cost optimization, and reliability concerns.

The Case for Hybrid Infrastructure

Running everything in the cloud introduces single points of failure and unpredictable latency. Running everything locally limits your reasoning capabilities and context window. The hybrid approach gives you the best of both worlds.

Local Systems (Latency-Sensitive Tasks)

- Market data ingestion and WebSocket management

- Technical indicator calculations (EMA, RSI, MACD, VWAP, ATR)
- Signal generation and scoring
- Order execution logic and position management
- Local LLM inference for fast classification tasks

Cloud AI Models (Reasoning Tasks)

- Strategic analysis and multi-factor evaluation
- Complex reasoning with large context windows
- Research workflows and document analysis
- Large-context data summarization and pattern recognition
- RAG-enhanced decision making

Architecture Diagram

The following table illustrates the data flow through the hybrid system:

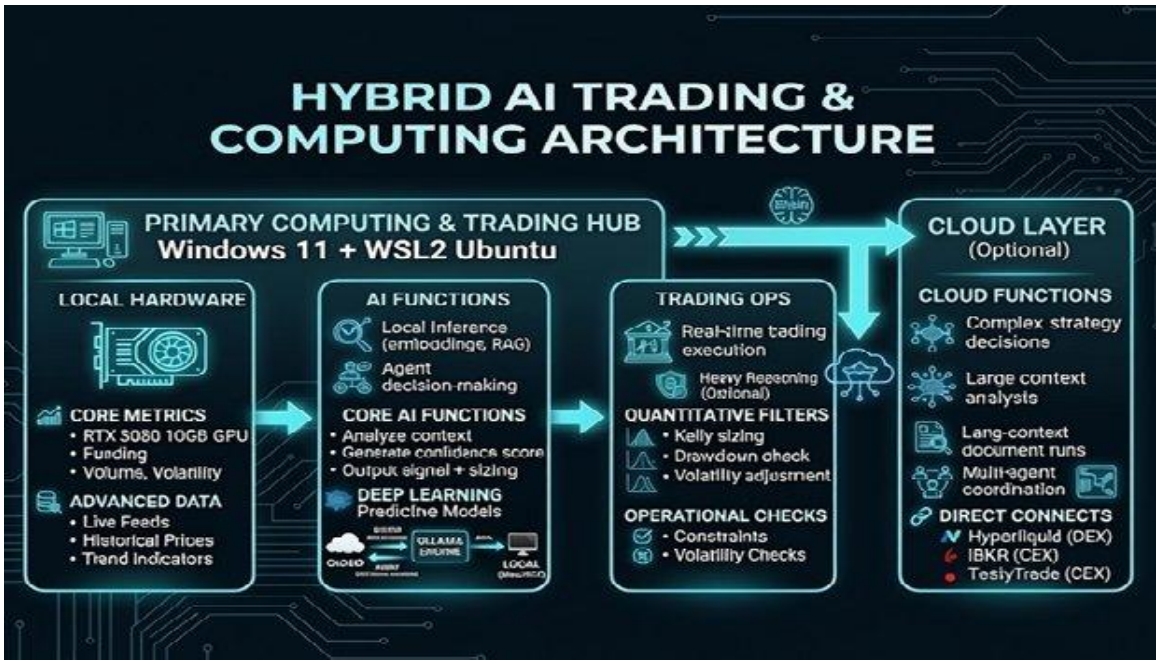
Layer	Component	Location	Latency Target
Data Ingestion	WebSocket Feeds	Local	< 10ms
Signal Generation	Strategy Engine	Local	< 50ms
AI Evaluation	LLM Reasoning	Local + Cloud	< 2s
Risk Check	Risk Engine	Local	< 10ms
Execution	Order Router	Local	< 100ms
Monitoring	Dashboards + Alerts	Local + Cloud	< 5s

Communication Patterns

Components communicate via internal message queues, REST APIs, or direct function calls depending on latency requirements. For ultra-low-latency paths (data ingestion to execution), use in-process function calls. For monitoring and alerting, HTTP-based APIs are acceptable.

```
# Example: Internal message queue pattern using asyncio
import asyncio
from collections import deque

class SignalBus:
    def __init__(self):
        self._queues: dict[str, asyncio.Queue] = {}

    def subscribe(self, topic: str) -> asyncio.Queue:
        if topic not in self._queues:
            self._queues[topic] = asyncio.Queue()
        return self._queues[topic]

    async def publish(self, topic: str, message: dict):
        if topic in self._queues:
            await self._queues[topic].put(message)

# Usage
bus = SignalBus()
signal_queue = bus.subscribe('trading_signals')
await bus.publish('trading_signals', {
    'symbol': 'BTC-USD',
    'direction': 'long',
    'score': 0.85,
    'timestamp': time.time()
})
```

Chapter 3: Windows + WSL Trading Infrastructure



A Windows workstation running WSL2 (Windows Subsystem for Linux) provides the most flexible environment for building AI trading systems. You get native GPU acceleration via CUDA, a full Linux development environment, and compatibility with the entire Python/Node.js trading ecosystem.

3.1 Recommended Hardware Specifications

Component	Minimum	Recommended
CPU	8-core (Ryzen 7 / i7)	16-core (Ryzen 9 / i9)
RAM	32 GB DDR4	64 GB DDR5
GPU	RTX 3080 (10 GB VRAM)	RTX 4090 (24 GB VRAM)
Storage	512 GB NVMe SSD	2 TB NVMe SSD
Network	Stable broadband	Low-latency wired connection

3.2 Installing WSL2

WSL2 runs a real Linux kernel inside a lightweight VM, giving you full Linux compatibility with near-native performance. Open PowerShell as Administrator and run:

```
# PowerShell (Run as Administrator)
wsl --install -d Ubuntu-24.04

# Verify installation
wsl --list --verbose

# Set WSL2 as default (if not already)
wsl --set-default-version 2
```

After installation, launch Ubuntu from the Start menu and create your Linux user account. Then update the system:

```
# Inside WSL2 Ubuntu
sudo apt update && sudo apt upgrade -y
sudo apt install -y build-essential git curl wget unzip \
    software-properties-common apt-transport-https \
    python3-pip python3-venv python3-dev \
    nodejs npm jq htop tmux
```

3.3 NVIDIA GPU Setup for WSL2

CUDA support in WSL2 requires the Windows NVIDIA driver (not a Linux driver). The Windows driver exposes GPU capabilities to the WSL2 kernel automatically.

1. Install the latest NVIDIA Game Ready or Studio driver on Windows from nvidia.com/drivers
2. Verify GPU visibility inside WSL2:

```
# Inside WSL2
nvidia-smi

# Expected output shows your GPU model, driver version, CUDA version
# Example:
# NVIDIA-SMI 560.xx  Driver Version: 560.xx  CUDA Version: 12.6
```

3. Install CUDA toolkit inside WSL2 (for compiling CUDA code):

```
# Install CUDA toolkit (WSL2 version)
wget https://developer.download.nvidia.com/compute/cuda/repos/wsl-
ubuntu/x86_64/cuda-keyring_1.1-1_all.deb
sudo dpkg -i cuda-keyring_1.1-1_all.deb
sudo apt update
sudo apt install -y cuda-toolkit-12-6

# Add to PATH
echo 'export PATH=/usr/local/cuda/bin:$PATH' >> ~/.bashrc
echo 'export LD_LIBRARY_PATH=/usr/local/cuda/lib64:$LD_LIBRARY_PATH' >> ~/.bashrc
source ~/.bashrc

# Verify
nvcc --version
```

> **Important:**
> Do NOT install NVIDIA Linux drivers inside WSL2. Only install the Windows driver. The WSL2 kernel handles GPU passthrough automatically.

3.4 Python Environment Setup

Use virtual environments to isolate your trading projects. This prevents dependency conflicts between different bots and tools.

```
# Create a project directory structure
mkdir -p ~/trading/{bots,data,models,logs,configs}
cd ~/trading

# Create and activate a virtual environment
python3 -m venv .venv
source .venv/bin/activate

# Install core trading dependencies
pip install --upgrade pip setuptools wheel
pip install \
    numpy pandas scipy scikit-learn \
    ccxt websockets aiohttp requests \
    python-dotenv pydantic \
    ta-lib pandas-ta \
    torch torchvision --index-url https://download.pytorch.org/whl/cu121 \
    transformers sentence-transformers \
    chromadb openai anthropic \
    ib_insync

# Verify PyTorch GPU access
python -c "import torch; print(f'CUDA available: {torch.cuda.is_available()}'); \
  print(f'GPU: {torch.cuda.get_device_name(0)}')"
```

> **Pro Tip:**
> Pin your dependency versions in a requirements.txt file. Trading bots should never break due to an unexpected package update. Use pip freeze > requirements.txt after confirming everything works.

3.5 Installing Ollama for Local LLM Inference

Ollama provides a simple interface for running large language models locally. Your GPU handles inference, keeping sensitive trading logic off third-party servers and eliminating API costs for high-frequency decisions.

```
# Install Ollama (run from WSL2)
curl -fsSL https://ollama.com/install.sh | sh

# Start the Ollama server
ollama serve &

# Pull models for trading use cases
ollama pull qwen2.5:14b        # Primary reasoning model
ollama pull nomic-embed-text    # Embeddings for RAG
ollama pull llama3.1:8b         # Fast classification

# Verify models are available
ollama list

# Test inference
curl http://localhost:11434/api/generate -d '{
  "model": "qwen2.5:14b",
  "prompt": "Analyze BTC market structure in 2 sentences.",
  "stream": false
}'
```

For production trading, configure Ollama to start automatically:

```
# Create a systemd service for Ollama (inside WSL2)
sudo tee /etc/systemd/system/ollama.service << 'EOF'
[Unit]
Description=Ollama LLM Server
After=network.target

[Service]
Type=simple
ExecStart=/usr/local/bin/ollama serve
Restart=always
RestartSec=5
Environment=OLLAMA_HOST=0.0.0.0:11434
Environment=OLLAMA_NUM_PARALLEL=4

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl enable ollama
sudo systemctl start ollama
```

3.6 Node.js and TypeScript Setup

Many trading venue SDKs and WebSocket clients are written in TypeScript/JavaScript. Set up a modern Node.js environment:

```
# Install Node.js via nvm (recommended over apt)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.0/install.sh | bash
source ~/.bashrc
nvm install 20
nvm use 20
```

```
# Install global tools
npm install -g typescript ts-node pm2 nodemon

# pm2 is critical for keeping trading bots alive
pm2 startup    # Configure auto-start on boot

# Example: Start a trading bot with pm2
pm2 start bot.js --name 'hyperliquid-scalper' --max-restarts 10
pm2 save        # Save process list for auto-restart
```

3.7 WSL2 Networking Configuration

WSL2 uses a NAT-based network by default, which can complicate WebSocket connections and inter-machine communication. Here is how to configure networking for trading use cases:

```
# Check your WSL2 IP address
hostname -I

# To expose WSL2 services to your LAN, use port forwarding
# Run this in PowerShell (Admin) on Windows:
netsh interface portproxy add v4tov4 \
    listenport=11434 listenaddress=0.0.0.0 \
    connectport=11434 connectaddress=$(wsl hostname -I | awk '{print $1}')

# For persistent access across reboots, consider Tailscale
# Install Tailscale in WSL2:
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up

# Now your WSL2 instance is accessible via Tailscale IP
# from any device on your tailnet (Mac, phone, etc.)
```

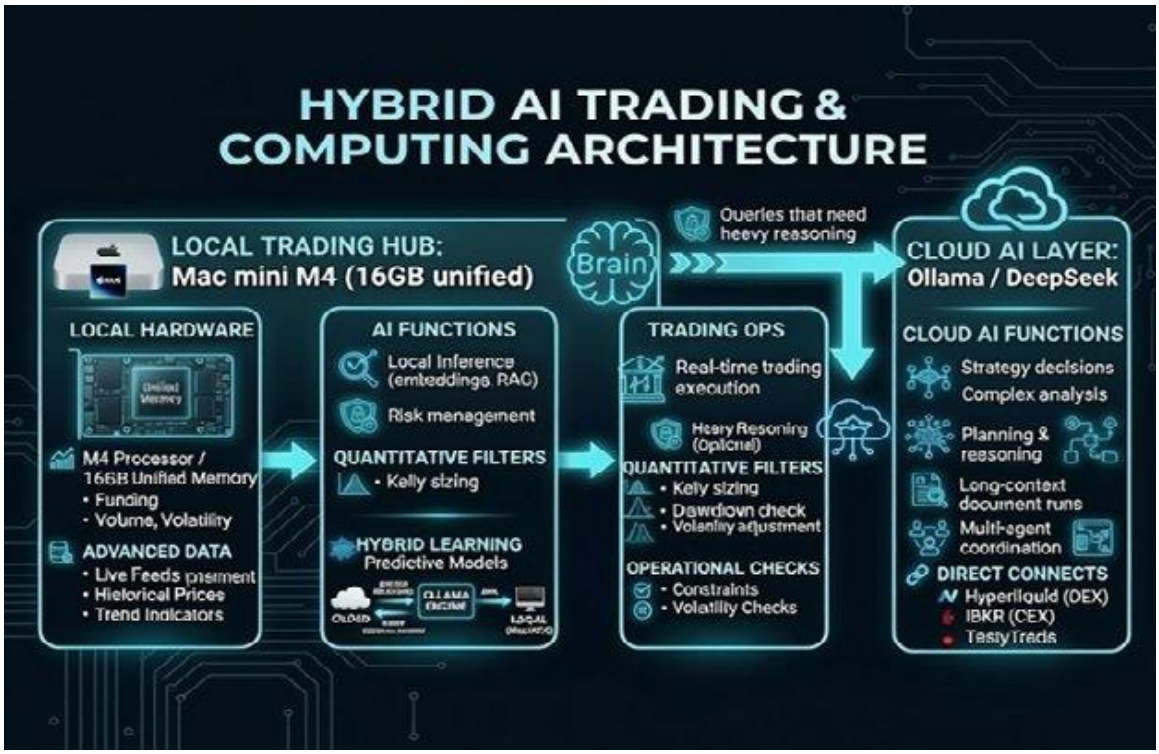
> **Tailscale Tip:**
> Tailscale Funnel can expose specific services (like your Ollama API or monitoring dashboards) to the public internet with automatic HTTPS. Perfect for mobile monitoring of your trading bots.

3.8 SSH Access Between Windows and WSL2

For remote management and IDE integration, configure SSH access:

```
# Install and configure SSH server in WSL2
sudo apt install -y openssh-server
sudo sed -i 's/#Port 22/Port 2222/' /etc/ssh/sshd_config
sudo sed -i 's/#PasswordAuthentication yes/PasswordAuthentication yes/'
/etc/ssh/sshd_config
sudo service ssh start

# Connect from Windows Terminal or VS Code:
ssh user@localhost -p 2222

# For VS Code Remote SSH, add to ~/.ssh/config on Windows:
# Host wsl-trading
#     HostName localhost
#     Port 2222
#     User your_username
```

Chapter 4: Mac AI Trading Infrastructure



Apple Silicon machines (M1 Pro/Max/Ultra, M2, M3, M4) provide exceptional energy efficiency and unified memory for AI workloads. A Mac serves as an excellent companion node for research, embeddings generation, strategy experimentation, and monitoring.

4.1 Why Apple Silicon for AI Trading

Apple Silicon uses a unified memory architecture where CPU, GPU, and Neural Engine share the same memory pool. This eliminates the bottleneck of copying data between CPU and GPU memory, which is particularly beneficial for large model inference.

Spec	Mac Studio M4 Max	Best For
Unified Memory	Up to 128 GB	Loading large models (70B+)
GPU Cores	40 GPU cores	Parallel inference tasks
Neural Engine	16-core	ML acceleration
Power	~75W under load	24/7 operation, low electricity cost
Noise	Near-silent	Home office / always-on server

4.2 macOS Environment Setup

Start by installing the essential developer tools and package manager:

```
# Install Xcode Command Line Tools
xcode-select --install
```

```
# Install Homebrew
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Add Homebrew to PATH (Apple Silicon)
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"

# Install essential tools
brew install python@3.12 node git wget jq tmux htop
brew install --cask iterm2 visual-studio-code
```

4.3 Python Environment for Mac

Set up an isolated Python environment optimized for Apple Silicon:

```
# Create project structure
mkdir -p ~/trading/{bots,data,models,logs,configs}
cd ~/trading

# Create virtual environment
python3 -m venv .venv
source .venv/bin/activate

# Install core packages (Apple Silicon optimized)
pip install --upgrade pip setuptools wheel
pip install \
    numpy pandas scipy scikit-learn \
    ccxt websockets aiohttp requests \
    python-dotenv pydantic \
    pandas-ta \
    torch torchvision \
    transformers sentence-transformers \
    chromadb openai anthropic \
    mlx mlx-lm

# Verify Metal (Apple GPU) acceleration
python -c "import torch; print(f'MPS available:
{torch.backends.mps.is_available()}')"

# Verify MLX (Apple's native ML framework)
python -c "import mlx.core as mx; print(f'MLX device: {mx.default_device()}')"
```

> **MLX Tip:**
> Apple's MLX framework is purpose-built for Apple Silicon and often outperforms PyTorch for inference on M-series chips. Consider using mlx-lm for local model inference as an alternative to Ollama for Mac-native performance.

4.4 Ollama on macOS

Ollama runs natively on macOS with Apple Silicon GPU acceleration:

```
# Install Ollama
brew install ollama

# Start Ollama (runs as a background service)
brew services start ollama

# Pull models
ollama pull qwen2.5:14b
ollama pull nomic-embed-text
ollama pull llama3.1:8b

# Configure for network access (other machines on tailnet)
# Edit ~/Library/LaunchAgents/com.ollama.ollama.plist
# Or set environment variable:
launchctl setenv OLLAMA_HOST 0.0.0.0:11434

# Restart to apply
brew services restart ollama

# Verify from another machine:
curl http://<mac-tailscale-ip>:11434/api/tags
```

4.5 Tailscale for Secure Multi-Node Communication

Tailscale creates a secure mesh VPN between your Windows trading workstation and Mac research node, enabling seamless communication without exposing services to the public internet:

```
# Install Tailscale on Mac
brew install --cask tailscale
# Or install from the Mac App Store

# After connecting, your devices can reach each other:
# Windows WSL2: 100.x.x.x (e.g., windows-trader)
# Mac Studio:   100.y.y.y (e.g., mac-research)

# Example: Mac queries Windows Ollama for heavy GPU inference
curl http://windows-trader:11434/api/generate -d '{
  "model": "qwen2.5:14b",
  "prompt": "Analyze current BTC market regime.",
  "stream": false
}'

# Example: Windows bot queries Mac for embeddings
curl http://mac-research:11434/api/embeddings -d '{
  "model": "nomic-embed-text",
  "prompt": "BTC broke through 100k resistance with high volume"
}'
```

4.6 Mac as a Monitoring Dashboard

The Mac makes an excellent always-on monitoring node due to its low power consumption and silent operation. Configure it to run monitoring dashboards and alert systems:

```
# Install monitoring tools
pip install grafana-client prometheus-client flask

# Simple trading monitor with Flask
# monitors/dashboard.py
from flask import Flask, jsonify
import requests

app = Flask(__name__)

@app.route('/status')
def status():
    # Query each bot's health endpoint
    bots = {
        'hyperliquid': 'http://windows-trader:8081/health',
        'polymarket': 'http://windows-trader:8082/health',
        'mes-options': 'http://windows-trader:8083/health',
    }
    statuses = {}
    for name, url in bots.items():
        try:
            r = requests.get(url, timeout=5)
            statuses[name] = r.json()
        except Exception as e:
            statuses[name] = {'status': 'DOWN', 'error': str(e)}
    return jsonify(statuses)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Chapter 5: Strategy Engine

The strategy engine is the quantitative core of your trading platform. It transforms raw market data into scored trading signals that flow to the AI decision layer for evaluation.

5.1 Strategy Types

Mean Reversion

Markets often revert toward equilibrium after extreme price movements. Mean reversion strategies identify overbought or oversold conditions using indicators like RSI, Bollinger Bands, and z-scores.

```
import pandas as pd
import numpy as np

class MeanReversionStrategy:
    def __init__(self, lookback=20, z_threshold=2.0):
        self.lookback = lookback
        self.z_threshold = z_threshold

    def generate_signal(self, prices: pd.Series) -> dict:
        """Generate mean reversion signal."""
        mean = prices.rolling(self.lookback).mean().iloc[-1]
        std = prices.rolling(self.lookback).std().iloc[-1]
        current = prices.iloc[-1]
        z_score = (current - mean) / std if std > 0 else 0

        if z_score < -self.z_threshold:
            return {'direction': 'long', 'score': min(abs(z_score) / 3, 1.0),
                    'strategy': 'mean_reversion', 'z_score': z_score}
        elif z_score > self.z_threshold:
```

```
            return {'direction': 'short', 'score': min(abs(z_score) / 3, 1.0),
                    'strategy': 'mean_reversion', 'z_score': z_score}
        return {'direction': 'neutral', 'score': 0, 'strategy': 'mean_reversion'}
```

Momentum Breakouts

Strong directional moves can produce sustained trends. Momentum strategies detect breakouts using volume confirmation, price action, and multi-timeframe analysis.

```
class MomentumBreakoutStrategy:
    def __init__(self, ema_fast=12, ema_slow=26, volume_mult=1.5):
        self.ema_fast = ema_fast
        self.ema_slow = ema_slow
        self.volume_mult = volume_mult

    def generate_signal(self, df: pd.DataFrame) -> dict:
        """Detect momentum breakout with volume confirmation."""
        ema_f = df['close'].ewm(span=self.ema_fast).mean()
        ema_s = df['close'].ewm(span=self.ema_slow).mean()
        avg_vol = df['volume'].rolling(20).mean()

        # EMA crossover with volume spike
        cross_up = (ema_f.iloc[-1] > ema_s.iloc[-1]) and \
                   (ema_f.iloc[-2] <= ema_s.iloc[-2])
        cross_down = (ema_f.iloc[-1] < ema_s.iloc[-1]) and \
                     (ema_f.iloc[-2] >= ema_s.iloc[-2])
        vol_confirm = df['volume'].iloc[-1] > avg_vol.iloc[-1] * self.volume_mult

        if cross_up and vol_confirm:
            return {'direction': 'long', 'score': 0.8, 'strategy': 'momentum'}
        elif cross_down and vol_confirm:
            return {'direction': 'short', 'score': 0.8, 'strategy': 'momentum'}
        return {'direction': 'neutral', 'score': 0, 'strategy': 'momentum'}
```

Funding Rate Arbitrage

In perpetual futures markets, extreme funding rates signal crowded positioning. When funding is extremely positive (longs paying shorts), contrarian short positions can be profitable.

Options Income

Selling option premium during range-bound conditions generates steady income. Strategies like credit spreads and iron condors benefit from theta decay. This works especially well with MES micro futures options on Interactive Brokers.

```
# MES Options: Directional Vertical Debit Spread
from ib_insync import IB, Option, MarketOrder, LimitOrder

class MESOptionsStrategy:
    def __init__(self, ib: IB):
        self.ib = ib
        self.symbol = 'MES'

    async def bull_call_spread(self, expiry: str,
                               long_strike: float,
                               short_strike: float,
                               quantity: int = 1):
        """Enter a bullish vertical debit spread."""
        long_call = Option(self.symbol, expiry, long_strike, 'C', 'CME')
        short_call = Option(self.symbol, expiry, short_strike, 'C', 'CME')
```

```
        # Qualify contracts
        [long_call] = self.ib.qualifyContracts(long_call)
        [short_call] = self.ib.qualifyContracts(short_call)

        # Create combo order
        combo = self.ib.createComboLegs([
            (long_call.conId, 'BUY'),
            (short_call.conId, 'SELL'),
        ])
        order = LimitOrder('BUY', quantity, max_debit)
        trade = self.ib.placeOrder(combo, order)
        return trade
```

5.2 Multi-Strategy Signal Aggregation

Running multiple strategies simultaneously and aggregating their signals produces more robust trading decisions. A weighted ensemble approach helps filter noise:

```
class SignalAggregator:
    def __init__(self, strategies: dict):
        """strategies: {'name': (strategy_instance, weight)}"""
        self.strategies = strategies

    def aggregate(self, market_data) -> dict:
        """Combine signals from all strategies."""
        long_score = 0
        short_score = 0
        total_weight = sum(w for _, w in self.strategies.values())

        for name, (strategy, weight) in self.strategies.items():
            signal = strategy.generate_signal(market_data)
            normalized_weight = weight / total_weight
            if signal['direction'] == 'long':
                long_score += signal['score'] * normalized_weight
            elif signal['direction'] == 'short':
                short_score += signal['score'] * normalized_weight

        if long_score > short_score and long_score > 0.5:
            return {'direction': 'long', 'score': long_score}
        elif short_score > long_score and short_score > 0.5:
            return {'direction': 'short', 'score': short_score}
        return {'direction': 'neutral', 'score': 0}
```

Chapter 6: AI Decision Layer

Large Language Models serve as an intelligent filter between signal generation and order execution. The AI layer examines contextual information that pure quantitative models may miss, including volatility regime, correlated asset behavior, historical patterns, and market sentiment.

6.1 Signal Evaluation Pipeline

Every trading signal passes through the AI layer before execution:

Market Data → Strategy Signal → AI Evaluation → Risk Filter → Execution

```
import json
import aiohttp

class AIDecisionLayer:
    def __init__(self, ollama_url='http://localhost:11434',
                 model='qwen2.5:14b'):
        self.ollama_url = ollama_url
        self.model = model

    async def evaluate_signal(self, signal: dict,
                              market_context: dict) -> dict:
        """Use LLM to evaluate a trading signal."""
        prompt = f"""You are a quantitative trading analyst.
Evaluate this trading signal and respond with JSON only.

Signal: {json.dumps(signal)}
Market Context: {json.dumps(market_context)}

Respond with: {{
  "approved": true/false,
  "confidence": 0.0-1.0,
  "reasoning": "brief explanation",
  "position_size_modifier": 0.5-1.5
}}"""

        async with aiohttp.ClientSession() as session:
            resp = await session.post(
                f'{self.ollama_url}/api/generate',
                json={'model': self.model, 'prompt': prompt,
                      'stream': False, 'format': 'json'}
            )
            result = await resp.json()
            return json.loads(result['response'])
```

6.2 Cloud Fallback for Complex Analysis

For high-stakes decisions or complex multi-factor analysis, route to cloud models (Claude, GPT-4) with larger context windows:

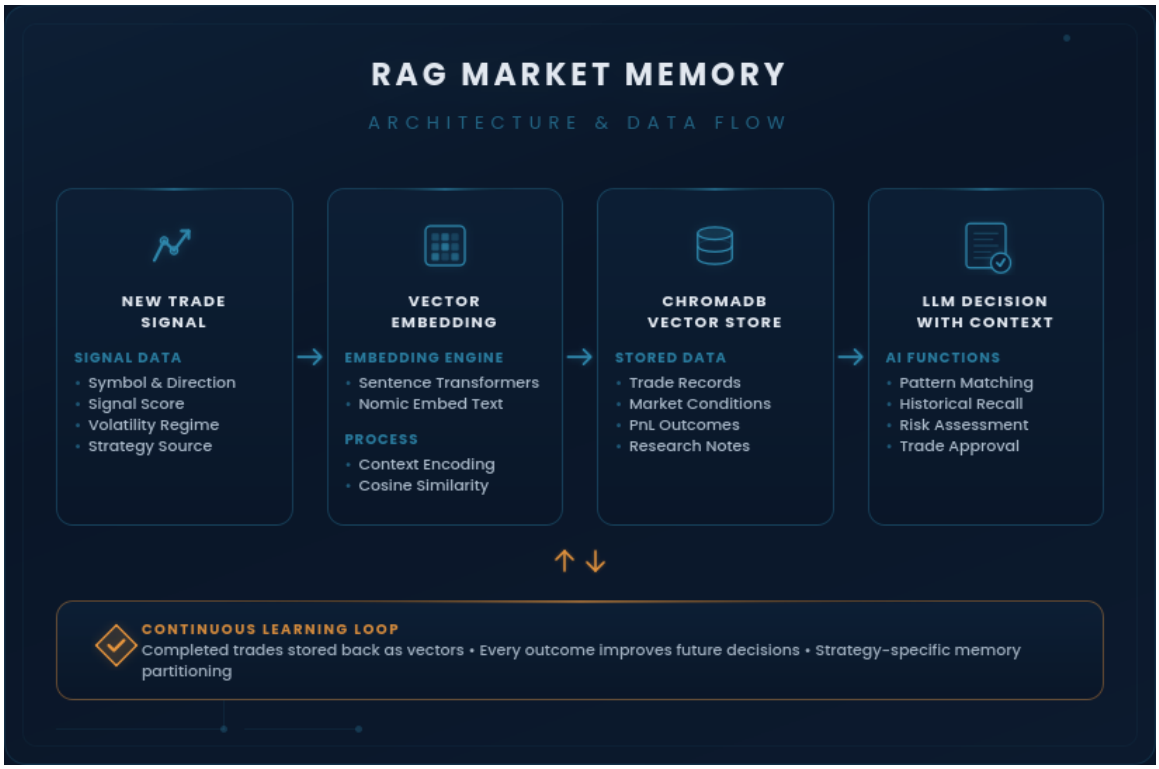
```
from anthropic import Anthropic

class CloudDecisionLayer:
    def __init__(self):
        self.client = Anthropic()  # Uses ANTHROPIC_API_KEY env var

    async def deep_analysis(self, signal: dict,
                            historical_context: str,
                            news_summary: str) -> dict:
        """Use Claude for complex multi-factor analysis."""
        message = self.client.messages.create(
            model='claude-sonnet-4-20250514',
            max_tokens=1024,
            messages=[{
                'role': 'user',
                'content': f"""Analyze this trade setup.

Signal: {json.dumps(signal)}
Historical patterns: {historical_context}
Recent news: {news_summary}

Provide: approval, confidence 0-1, risk assessment,
and position size recommendation as JSON."""
            }]
        )
        return json.loads(message.content[0].text)
```

6.3 Tiered Model Routing

Implement intelligent routing that selects the appropriate model based on signal importance and market conditions:

Signal Score	Model	Location	Rationale
0.5 - 0.65	llama3.1:8b	Local	Fast reject/approve for marginal signals
0.65 - 0.80	qwen2.5:14b	Local	Balanced reasoning for medium signals
0.80 - 1.00	Claude Sonnet	Cloud	Deep analysis for high-conviction trades

Chapter 7: RAG Market Memory



Retrieval-Augmented Generation allows your trading system to incorporate historical knowledge. Trade records, market conditions, and research notes are embedded into vectors and stored in a database. When evaluating a new signal, the system retrieves similar historical scenarios and provides them to the AI model for context.

7.1 Setting Up ChromaDB for Trade Memory

```
import chromadb
from sentence_transformers import SentenceTransformer
import json, time

class TradeMemory:
    def __init__(self, persist_dir='./data/chroma'):
        self.client = chromadb.PersistentClient(path=persist_dir)
        self.collection = self.client.get_or_create_collection(
            name='trade_history',
            metadata={'hnsw:space': 'cosine'}
        )
        self.embedder = SentenceTransformer('all-MiniLM-L6-v2')

    def store_trade(self, trade: dict):
        """Store a completed trade with its context."""
        text = (f"{trade['symbol']} {trade['direction']} "
                f"entry={trade['entry_price']} exit={trade['exit_price']} "
                f"pnl={trade['pnl']} regime={trade['volatility_regime']} "
                f"strategy={trade['strategy']} {trade['notes']}")
        embedding = self.embedder.encode(text).tolist()
        self.collection.add(
            ids=[f"trade_{int(time.time())}_{trade['symbol']}"],
```

```
            embeddings=[embedding],
            documents=[text],
            metadatas=[trade]
        )

    def recall_similar(self, context: str, n=5) -> list:
        """Retrieve similar historical scenarios."""
        embedding = self.embedder.encode(context).tolist()
        results = self.collection.query(
            query_embeddings=[embedding], n_results=n
        )
        return list(zip(
            results['documents'][0],
            results['metadatas'][0]
        ))
```

7.2 Integrating Memory with AI Decisions

When a new signal arrives, query the memory for similar past scenarios and include them in the LLM prompt:

```
async def evaluate_with_memory(self, signal, market_context):
    # Build context string
    context = (f"{signal['symbol']} {signal['direction']} "
               f"score={signal['score']:.2f} "
               f"regime={market_context['regime']}")

    # Retrieve similar historical trades
    similar = self.memory.recall_similar(context, n=5)
    history_text = '\n'.join([
        f"Past trade: {doc} -> PnL: {meta.get('pnl', 'N/A')}"
        for doc, meta in similar
    ])

    # Include history in AI evaluation
    prompt = f"""Evaluate this signal with historical context.
Signal: {json.dumps(signal)}
Similar past trades:
{history_text}

Based on how similar setups performed historically,
should we take this trade?"""
    return await self.ai_layer.evaluate(prompt)
```

Chapter 8: Hyperliquid Trading Bot Architecture

Hyperliquid is a decentralized perpetual futures exchange designed for high-performance trading. Its on-chain order book and sub-second finality make it well-suited for algorithmic strategies.

8.1 Bot Architecture

A typical Hyperliquid trading bot consists of five interconnected components that run continuously:

Component	Responsibility
Market Data Collector	WebSocket feeds for real-time price, order book, and trade data
Strategy Engine	Technical analysis, signal generation, multi-strategy aggregation
AI Decision Layer	LLM-based signal evaluation and contextual analysis
Risk Manager	Position sizing, exposure limits, drawdown monitoring
Execution Engine	Order placement, fill management, position tracking

8.2 Hyperliquid API Integration

```
// TypeScript: Hyperliquid WebSocket + REST client
import { HyperliquidAPI } from 'hyperliquid-ts';

class HyperliquidBot {
  private api: HyperliquidAPI;
  private positions: Map<string, Position> = new Map();

  constructor(privateKey: string) {
    this.api = new HyperliquidAPI({ privateKey });
  }

  async placeOrder(symbol: string, side: 'buy' | 'sell',
                   size: number, price: number) {
    const order = await this.api.exchange.placeOrder({
      coin: symbol,
      is_buy: side === 'buy',
      sz: size,
      limit_px: price,
      order_type: { limit: { tif: 'Gtc' } },
      reduce_only: false,
    });
    return order;
  }

  async subscribeToMarketData(symbols: string[]) {
    for (const symbol of symbols) {
      this.api.subscriptions.subscribeToTrades(symbol, (trades) => {
        this.onTrade(symbol, trades);
      });
    }
  }
}
```

8.3 Scalper Configuration

A well-tuned scalper configuration targets high-liquidity assets with tight spreads. The following configuration has been battle-tested:

Parameter	Scalp Mode	Swing Mode (1hr)
Assets	BTC, ETH, XRP	BTC, ETH, SOL, XRP, PAXG
Min Signal Score	0.70	0.65
Position Sizing	Signal-strength dependent	Fixed % of equity
Max Leverage	5x	3x
Stop Loss	ATR-based trailing	Fixed % from entry

Chapter 9: Market Maker Strategy

Market makers provide liquidity by placing buy and sell orders around the mid-price, profiting from the bid-ask spread. This is one of the most consistent strategies in crypto perps markets when properly managed.

9.1 Core Market Making Logic

```
class MarketMaker:
    def __init__(self, spread_bps=10, max_inventory=100,
                 order_size=1.0, skew_factor=0.5):
        self.spread_bps = spread_bps
        self.max_inventory = max_inventory
        self.order_size = order_size
        self.skew_factor = skew_factor
        self.inventory = 0.0

    def compute_quotes(self, mid_price: float,
                       volatility: float) -> dict:
        """Compute bid/ask quotes with inventory skew."""
        # Widen spread during high volatility
        adjusted_spread = self.spread_bps * (1 + volatility * 2)
        half_spread = mid_price * adjusted_spread / 20000

        # Skew quotes based on inventory
        inventory_ratio = self.inventory / self.max_inventory
        skew = mid_price * inventory_ratio * self.skew_factor / 100

        bid = mid_price - half_spread - skew
        ask = mid_price + half_spread - skew

        # Reduce size when inventory is extreme
        bid_size = self.order_size * max(0.2, 1 - inventory_ratio)
        ask_size = self.order_size * max(0.2, 1 + inventory_ratio)

        return {
            'bid': round(bid, 2), 'bid_size': bid_size,
            'ask': round(ask, 2), 'ask_size': ask_size,
        }
```

Inventory management is critical. Excess long inventory should encourage selling (by lowering both bid and ask), while excess short inventory should encourage buying.

Chapter 10: Volatility Regime Detection

Market behavior changes dramatically across volatility regimes. Detecting the current regime allows your system to automatically switch between strategies: mean reversion in low volatility, momentum/breakout in high volatility.

10.1 Regime Classification

```
import numpy as np
import pandas as pd

class VolatilityRegimeDetector:
    def __init__(self, atr_period=14, lookback=100):
        self.atr_period = atr_period
        self.lookback = lookback

    def classify(self, df: pd.DataFrame) -> str:
        """Classify current volatility regime."""
        # Calculate ATR
        high_low = df['high'] - df['low']
        high_close = abs(df['high'] - df['close'].shift())
        low_close = abs(df['low'] - df['close'].shift())
        true_range = pd.concat([high_low, high_close, low_close],
                               axis=1).max(axis=1)
        atr = true_range.rolling(self.atr_period).mean()

        # Normalized ATR (as % of price)
        natr = (atr / df['close']) * 100

        # Percentile ranking
        current = natr.iloc[-1]
        percentile = (natr.tail(self.lookback) < current).mean()

        if percentile < 0.25:
            return 'LOW_VOL'
        elif percentile < 0.75:
            return 'NORMAL'
        else:
            return 'HIGH_VOL'
```

10.2 Strategy Selection by Regime

Regime	Active Strategies	Risk Adjustment
LOW_VOL	Mean Reversion, Options Income, Market Making	Normal position sizes, tight stops
NORMAL	All strategies active	Standard risk parameters
HIGH_VOL	Momentum Breakout, Reduced Market Making	Halved position sizes, wide stops

Chapter 11: Portfolio Risk Engine

Portfolio-level risk controls are your last line of defense against catastrophic losses. These controls operate independently of your strategy logic and can override any trading decision.

11.1 Risk Parameters

Control	Threshold	Action
Maximum Leverage	5x per position	Reject new orders
Max Exposure / Asset	20% of portfolio	Reduce position
Max Drawdown	10% from peak	Halt all trading
Daily Loss Limit	3% of portfolio	Close all positions
Correlation Limit	Max 3 correlated positions	Reject correlated trades

11.2 Kill Switch Implementation

```
class RiskEngine:
    def __init__(self, config: dict):
        self.max_drawdown = config.get('max_drawdown', 0.10)
        self.daily_loss_limit = config.get('daily_loss_limit', 0.03)
        self.max_leverage = config.get('max_leverage', 5.0)
        self.peak_equity = config.get('starting_equity')
        self.daily_start_equity = self.peak_equity
        self.halted = False

    def check(self, current_equity: float,
              proposed_trade: dict = None) -> dict:
        """Run all risk checks. Returns approval or rejection."""
        self.peak_equity = max(self.peak_equity, current_equity)
        drawdown = 1 - (current_equity / self.peak_equity)
        daily_pnl = (current_equity - self.daily_start_equity) \
                     / self.daily_start_equity

        # Kill switch: halt everything
        if drawdown >= self.max_drawdown:
            self.halted = True
            return {'approved': False, 'reason': 'MAX_DRAWDOWN',
                    'action': 'CLOSE_ALL_POSITIONS'}

        if daily_pnl <= -self.daily_loss_limit:
            self.halted = True
            return {'approved': False, 'reason': 'DAILY_LOSS_LIMIT',
                    'action': 'CLOSE_ALL_POSITIONS'}

        if self.halted:
            return {'approved': False, 'reason': 'TRADING_HALTED'}

        return {'approved': True}
```

Critical Safety Note:

Never disable the risk engine, even during backtesting. A bug that bypasses risk controls in production can result in total account liquidation. Always test your kill switch in a paper trading environment first.

Chapter 12: Monitoring and Observability

Automated trading systems must run continuously and reliably. Comprehensive monitoring infrastructure allows you to detect failures before they become costly.

12.1 What to Monitor

Metric	Alert Threshold	Channel
System Uptime	Any downtime > 30s	WhatsApp + Telegram
API Connectivity	3 consecutive failures	WhatsApp
Open Positions	Exceeds risk limits	WhatsApp + Dashboard
PnL	Daily loss > 2%	WhatsApp + Email
Execution Latency	> 500ms average	Dashboard
Model Inference	Ollama unresponsive	WhatsApp

12.2 WhatsApp Notification Integration

```
# notifications/whatsapp.py
import requests

class WhatsAppNotifier:
    def __init__(self, api_url: str, phone: str):
        self.api_url = api_url
        self.phone = phone

    def send(self, message: str, urgency: str = 'info'):
        """Send alert via WhatsApp."""
        prefix = {'critical': '\u26a0\ufe0f CRITICAL',
                  'warning': '\u26a0 WARNING',
                  'info': '\u2139\ufe0f INFO'}
        formatted = f"{prefix.get(urgency, '')}\n{message}"
        requests.post(self.api_url, json={
            'phone': self.phone, 'message': formatted
        })
```

12.3 Remote Command System

Enable remote control of your trading bots via WhatsApp or Telegram commands:

```
# Remote commands via webhook
COMMANDS = {
    '/status': 'Report current positions and PnL',
    '/halt': 'Halt all trading immediately',
    '/resume': 'Resume trading after halt',
    '/positions': 'List all open positions',
    '/close <symbol>': 'Close position for symbol',
    '/risk': 'Show current risk metrics',
}
```

Chapter 13: Scaling AI Trading Systems

Once your trading system becomes stable and profitable, scaling strategies allow you to increase capacity and diversify across more venues and strategies.

13.1 Scaling Strategies

- Running multiple strategy agents simultaneously across different timeframes
- Deploying separate research and execution nodes (Mac for research, Windows for execution)
- Using distributed compute with Tailscale mesh networking
- Implementing portfolio-level optimization engines that allocate capital across strategies
- Adding new venues: Polymarket prediction markets, additional CEX/DEX platforms

13.2 Multi-Agent Architecture with pm2

```
# ecosystem.config.js - pm2 process management
module.exports = {
  apps: [
    {
      name: 'hl-scalper',
      script: './bots/hyperliquid/scalper.js',
      max_restarts: 10,
      restart_delay: 5000,
      env: { NODE_ENV: 'production', VENUE: 'hyperliquid' }
    },
    {
      name: 'poly-btc',
      script: './bots/polymarket/btc_5min.js',
      max_restarts: 10,
      env: { NODE_ENV: 'production', VENUE: 'polymarket' }
    },
    {
      name: 'mes-options',
      script: './bots/ib/mes_spreads.py',
      interpreter: 'python3',
      max_restarts: 5,
      env: { VENUE: 'interactive_brokers' }
    },
    {
      name: 'risk-monitor',
      script: './monitors/risk_engine.py',
      interpreter: 'python3',
      max_restarts: 3,
    },
    {
      name: 'ollama-health',
      script: './monitors/ollama_health.js',
      cron_restart: '0 */6 * * *',
    }
  ]
};

// Deploy: pm2 start ecosystem.config.js
// Monitor: pm2 monit
// Logs: pm2 logs --lines 100
```

Chapter 14: Conclusion

AI-driven trading infrastructure combines quantitative models, machine learning systems, and automated execution engines into a unified platform. The architecture described in this playbook demonstrates how individual traders can build systems that resemble institutional trading infrastructure.

By combining hybrid AI infrastructure, quantitative strategies, LLM-based reasoning layers, automated execution, and portfolio risk controls, you can build an autonomous trading platform capable of adapting to evolving financial markets.

Key Takeaways

- Start with a solid hybrid infrastructure: local GPU for latency-sensitive tasks, cloud AI for complex reasoning
- Use tiered model routing to balance speed, cost, and reasoning quality
- Implement RAG market memory to learn from every trade
- Run multiple strategies and aggregate signals for more robust decisions
- Never compromise on risk management — the kill switch is your most important code
- Monitor everything and set up alerts for every failure mode you can imagine
- Scale gradually: add one new strategy or venue at a time

Next Steps

4. Set up your Windows + WSL2 or Mac development environment using Chapter 3 or 4
5. Implement the strategy engine from Chapter 5 with paper trading data
6. Configure Ollama and test the AI decision layer from Chapter 6
7. Build your first Hyperliquid bot using the architecture from Chapter 8
8. Implement the risk engine from Chapter 11 before going live
9. Set up monitoring and alerting from Chapter 12
10. Graduate to live trading with minimal position sizes

Good luck, and trade safely.

This playbook is for educational purposes. Trading involves risk. Past performance does not guarantee future results.